P4-Based Buffer Management Method for Throughput Fairness Improvement with Different Congestion Control Contention

Koshi Ono, Daiki Nobayashi, Takeshi Ikenaga
Kyushu Institute of Technology, Fukuoka, Japan
e-mail: ono.koshi266@mail.kyutech.jp, {nova, ike}@ecs.kyutech.ac.jp

2025 IEEE International Conference on Consumer Electronics (ICCE) | 979-8-3315-2116-5/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICCE63647.2025.10929998

***Abstract*—In TCP/IP, there are various types of congestion control algorithms, such as CUBIC and Bottleneck Bandwidth and Round-trip propagation time (BBR). When flows using CUBIC and BBR share a bottleneck link, it has been reported that fairness decreases with the buffer size of the relay node on the bottleneck link. To improve fairness in these circumstances, we have previously proposed a method assigning each congestion control algorithm a separate buffer at the relay node using a programmable data plane. We have also proposed a packet scheduling method, Fair Queuing with Holding Time (FQ-HT), that allows relay nodes to send packets fairly from multiple buffers by setting an appropriate holding time. The appropriate holding time depends on the bandwidth of the bottleneck link and the buffer size of the relay node. However, if the holding time is not set appropriately, the fairness may not be improved or the total throughput may be reduced. In this paper, we propose Fair Queuing with Variable Holding Time (FQ-VHT), which automatically sets the appropriate holding time based on the difference in the number of send packets between the buffers. Emulation experiments showed that FQ-VHT can set an appropriate holding time based on network conditions.**

***Index Terms*—Programmable data plane, P4, Congestion control, BBR, CUBIC**

I. Introduction

Network congestion occurs when demand for data delivery exceeds the network's capacity, and it is a major source of performance problems. In Transmission Control Protocol/Internet Protocol (TCP/IP), a number of congestion control algorithms exist to promote the most efficient use of the network and avoid congestion. According to the results of a recent Web access survey, the most commonly used congestion control algorithms are CUBIC TCP at 36% and Bottleneck Bandwidth and Round-trip propagation time (BBR) at 22% [1]. However, throughput fairness decreases when CUBIC and BBR flows share a bottleneck link [2]. If the buffer size of the bottleneck link is small, BBR occupies the bandwidth, and if it is large, CUBIC occupies the bandwidth.

To improve throughput fairness among different congestion controls, we have previously proposed a method that uses a programmable data plane to store packets in a buffer for each congestion control algorithm [3]. However, BBR sends packets so that they do not accumulate in the buffer. Thus, even if buffers are divided, Fair Queuing cannot sufficiently improve fairness. Therefore, we proposed Fair Queuing with Holding Time (FQ-HT) as a packet scheduling method to ensure fair transmission from each buffer [3]. We demonstrated that FQ-HT can achieve high fairness without decreasing throughput when an appropriate holding time is used. However, the appropriate holding time depends on the bandwidth and buffer size of the bottleneck link.

In this paper, we propose a packet scheduling scheme called Fair Queuing with Variable Holding Time (FQ-VHT), in which an appropriate holding time is determined and set according to network conditions. We evaluated the effectiveness of the proposed method by emulation. The remainder of this paper is organized as follows. Section 2 gives a brief review of related literature. Section 3 describes the existing packet scheduling method. In Sections 4 and 5, we present the proposed method and an evaluation of its performance using emulation. Finally, Section 6 concludes the paper.

II. Related Work

A number of approaches to ensure fairness among congestion control algorithms have been proposed. The methods for improving fairness described in this section focus on the control of sending terminals. Such methods require adjusting the control of each sending terminal, which may increase the load on the sending terminal. In this study, we propose a method to improve fairness at relay nodes when CUBIC and BBR conflict. This section gives a brief overview of related methods proposed in previous studies. Song et al. [4], [5] proposed the BBR-CWS (BBR Congestion Window Scaling) scheme to improve fairness between BBR and other congestion control algorithms. Using BBR-CWS instead of BBR with CUBIC can increase fairness and decrease the number of packet retransmissions. Song et al. [4], [5] also evaluated the performance of BBRv2. Their findings indicated that BBRv2 improves fairness for small buffers below 1 bandwidth-delay product (BDP) and when sharing bottleneck links, and in the latter case also prevents excessive packet loss. However, there are still some problems with large buffer sizes when contending with different round-trip time (RTT) flows and loss-based congestion control algorithms.

Utsumi et al. [6] proposed a model to estimate the average packet transmission rate of CUBIC when CUBIC and BBR compete on the same bottleneck link.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on February 04,2026 at 17:00:34 UTC from IEEE Xplore. Restrictions apply.

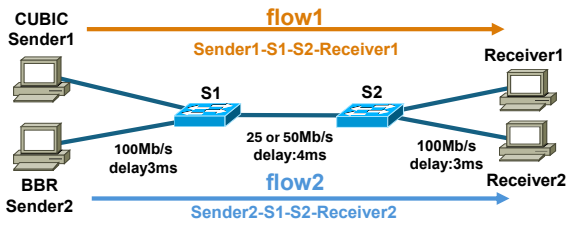


Fig. 1: Diagram of Experimental Environment

Ishikura et al. [7] found that when BBR contends with loss-based congestion control, the RTT is misestimated and network congestion increases due to the periodic queues in the buffers of the bottleneck link. They mitigated this problem by using priority packets to estimate and adjust RTT.

Fejes et al. [8] conducted experiments with various combinations of Active Queue Management (AQM) and congestion control algorithms to analyze flow fairness and coexistence. They reported that the latest active queue management algorithms, Proportional Integral Controller Enhanced and Global Synchronization Protection, do not support coexistence with modern congestion control. Even with AQM, it is difficult to improve fairness among different congestion controls with a single buffer.

Kfoury et al. [9] proposed a scheme to program relay nodes in P4 [10] and assign them to dedicated queues by identifying congestion control algorithms in real time. They reported improved fairness by separating buffers.

We aim to further improve fairness by proposing a method to retrieve packets from queues prepared for each congestion control algorithm.

III. Previous Work

Fair Queuing (FQ) is a typical packet scheduling method used to send packets from multiple buffers in a fair way. FQ sends packets preferentially from buffers with low amount of transmitted data., whereas BBR controls sending so as not to accumulate packets in buffers. This produces a situation where no packets exist in the buffer that stores BBR packets, resulting in a decrease in fairness.

A. Fair Queuing with Holding Time

As with FQ, Fair Queuing with Holding Time (FQ-HT) sends packets preferentially from buffers with low amount of sent data, but it also implements a holding time to address the above challenge. If there are no packets in the buffer from which packets are scheduled to be sent, the scheduler waits for packets to arrive for a set holding time. If no packets arrive during the holding time, the scheduler transitions to sending packets from the buffer with the next lowest number of send packets. By setting an appropriate holding time, FQ-HT can wait for BBR packets to arrive and improve fairness between BBR flows and other flows using congestion control. However, if the holding time is not sufficient, fairness is not improved, and if the holding time is too long, the total throughput is reduced. Therefore, it is necessary to set an appropriate holding time.

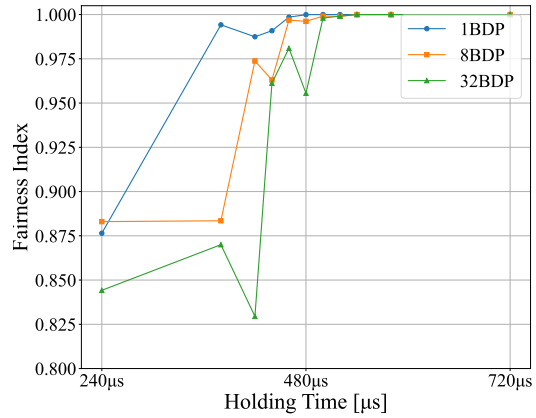


Fig. 2: Fairness Index for FQ-HT (25 Mb/s)

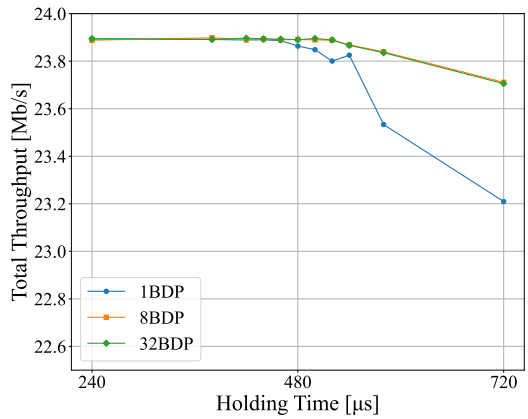


Fig. 3: Total Throughput for FQ-HT (25 Mb/s)

B. Preliminary Experiment

We performed an experiment to determine how the fairness and total throughput of FQ-HT relate to the holding time. Using the network emulator mininet [11], we configured the network shown in Figure 1. The relay node deployed a software switch BMv2 [12] that can execute the data plane programming language P4. The bottleneck link is between S1 and S2, and the bottleneck bandwidth was set to 25 and 50 Mb/s. Delays were set to 3 ms between the CUBIC and BBR Senders and S1, 3 ms between S2 and Receivers 1 and 2, and 4 ms between S1 and S2, so that the RTT between Sender and Receiver was 20 ms. In S1, separate buffers were designated for each congestion control algorithm, and packets were scheduled using FQ-HT. TCP flows were generated from each Sender to Receiver for 100 s using iperf. To determine the steady-state throughput, we measured the average throughput interval in the range of 20–100 s. The buffer size of the bottleneck link was set to 1, 8, and 32 BDP.

Figures 2 and 3 show the fairness and total throughput plotted against the holding time for a bottleneck link bandwidth

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on February 04,2026 at 17:00:34 UTC from IEEE Xplore. Restrictions apply.

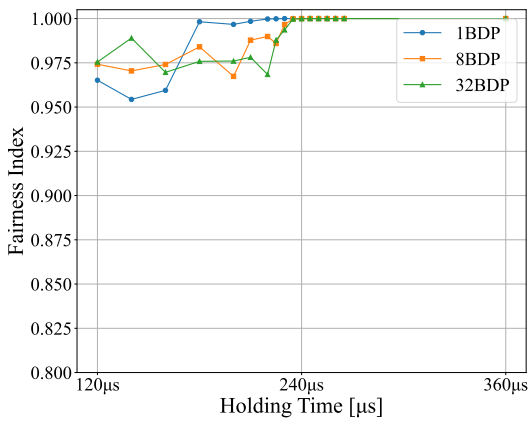


Fig. 4: Fairness Index for FQ-HT (50 Mb/s)

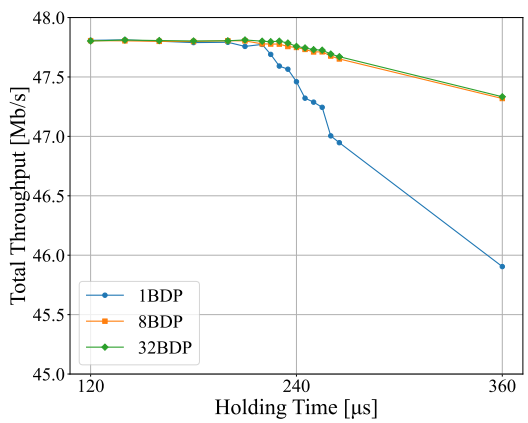


Fig. 5: Total Throughput for FQ-HT (50 Mb/s)

of 25 Mb/s. As shown in these figures, at a bottleneck link bandwidth of 25 Mb/s and buffer sizes of 1, 8, and 32 BDP, the fairness and total throughput are optimized at 460, 500, and 520 µs, respectively. This demonstrates than the appropriate holding time depends on the buffer size of the bottleneck link. When the bottleneck link bandwidth is 25 Mb/s, the sending time of one packet is 480 µs, which is close to the appropriate holding times of 460–520 µs indicated above. As shown in Figures 4 and 5, at a bottleneck link bandwidth is 50 Mb/s, the fairness and total throughput are optimized at 200 µs for a buffer size of 1 BDP and 235 µs for 8 and 32 BDP. When the bottleneck link bandwidth is 50 Mb/s, the sending time of one packet is 240 µs, which is once again close to the appropriate holding times of 200–235 µs above.

IV. Proposed Method

The preliminary experiment described in Section III-B demonstrated that the appropriate holding time depends on the buffer size of the bottleneck link and is close to the time required to send one packet on the bottleneck link. Therefore, we propose a modified FQ algorithm called Fair Queuing with Variable Holding Time (FQ-VHT) that can automatically set an appropriate holding time depending on the buffer size and bandwidth of the bottleneck link. In FQ-VHT, the holding time is determined by the following equation:

$$T_{\text{hold}} = rDT_{\text{send}}^2 \tag{1}$$

where D is the difference between the number of send packets in the two buffers, T_{send} is the time it takes for the bottleneck link to send one packet, and r is a constant. This formula sets the holding time to T_{send} in $\frac{1}{r}$ seconds if packets continue to be sent from one buffer.

When the difference in the number of send packets is large, the holding time increases, but when the holding time reaches an appropriate level, the packet difference becomes small. Therefore, FQ-VHT can prevent the holding time from becoming too high. We designed FQ-VHT to automatically set an appropriate holding time based on network conditions with the aim of optimizing fairness and total throughput.

V. Performance Evaluation Using Emulation

We evaluated the performance of FQ-VHT using network emulation to compare its fairness and throughput with those of other variations of FQ under different network conditions. We also observed how aspects of the network behavior changed over time to assess the functionality of the FQ-VHT algorithm. This section presents the results of our evaluation.

A. Experimental Environment

Emulation experiments were conducted to evaluate the performance of FQ-VHT. The experimental setup was the same as in the preliminary experiment described in Section III-B with the network topology shown in Figure 1. As before, each congestion control algorithm was assigned a separate buffer in S1. Three packet scheduling methods (FQ, FQ-HT, and FQ-VHT) were used to send packets from the buffers. The holding time in FQ-HT was set to the time required to send one packet in the bottleneck link bandwidth. The constant r in FQ-VHT (Eq. 1) was set to 1 and 0.2 to compare its impact on the performance. The fairness was evaluated using Jain's fairness index, which is given by

$$FairnessIndex = \frac{(\sum_{i=1}^{n} x_i)^2}{(n \times \sum_{i=1}^{n} x_i^2)'} \tag{2}$$

where n is the number of senders and x_i is the throughput of the ith flow.

B. Experimental Results and Discussion

In this section, we present the fairness and throughput results obtained in the emulation experiment for bottleneck bandwidths of 25 and 50 Mb/s. We also assess how the holding time and the throughput of the two control algorithm buffers vary over the course of the experiment. With this evaluation, we discuss how the variability in the holding time implemented by our proposed FQ-VHT algorithm impacts the fairness and total throughput.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on February 04,2026 at 17:00:34 UTC from IEEE Xplore. Restrictions apply.

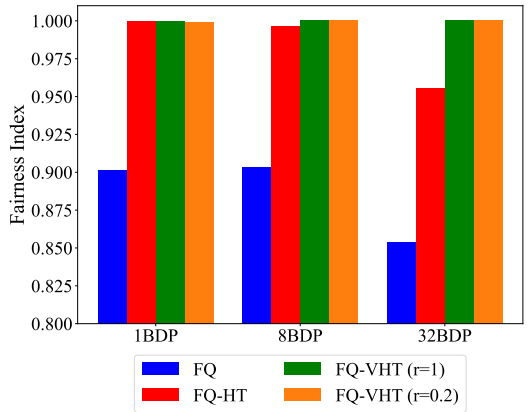
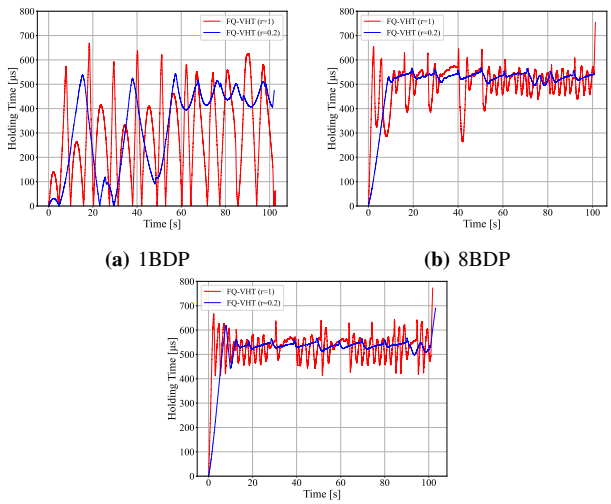


Fig. 6: Fairness Index (25 Mb/s)



(a) 1BDP **(b)** 8BDP

(c) 32BDP

Fig. 7: Time Variation of Holding Time (25 Mb/s)

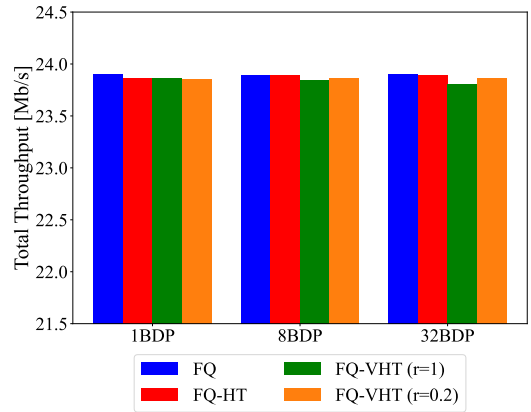
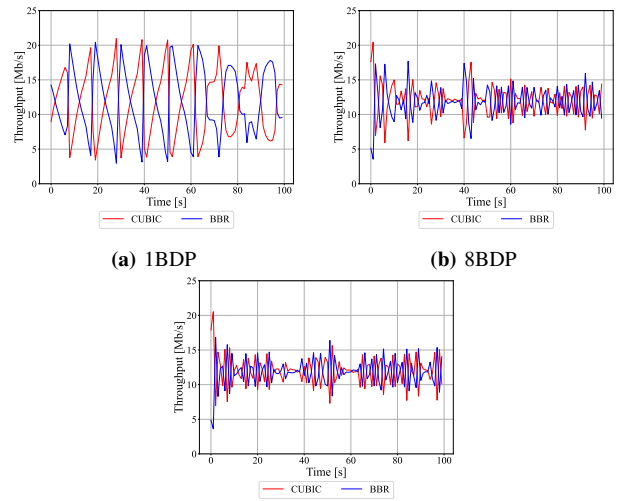


Fig. 8: Total Throughput (25 Mb/s)



(a) 1BDP **(b)** 8BDP

(c) 32BDP

Fig. 9: Time Variation of Throughput (25 Mb/s, r=1)

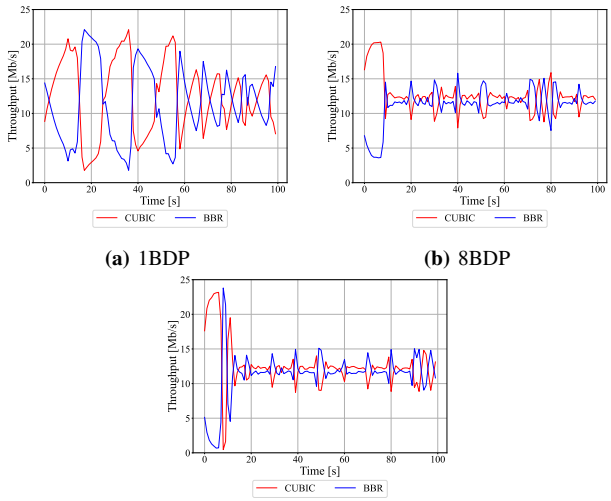
1) Bottleneck link bandwidth of 25 Mb/s: Figure 6 shows the fairness index of the three algorithms at different buffer sizes. The results demonstrate that FQ had a consistently lower fairness index than FQ-HT or FQ-VHT. Because BBR sends packets without buffering them, packets may not be sent, and FQ then cannot transmit fairly. In comparison to the proposed FQ-VHT algorithm, FQ-HT achieved a slightly lower fairness index at 8 BDP and 32 BDP. As shown in Figure 2, maximizing fairness at 25 Mb/s requires a holding time of 500 µs, which is longer than the time required to send one packet (480 µs). The fairness for FQ-VHT with both $r = 1$ and $r = 0.2$ was close to 1.

Figure 7 shows the FQ-VHT holding time plotted over the simulation time under different buffer sizes. As shown in Figure 7(a), at a buffer size of 1 BDP, the holding time quickly increased to approximately 500–600 s and then showed large oscillations. At 1 BDP, CUBIC is prone to packet loss due to buffer overflow, and its send rate was lower than at 8 and 32 BDP; thus, the send rate of CUBIC tends to often be less than that of BBR. Under these conditions, even if the holding time is short, it is still fair. At 8 and 32 BDP, the holding time fluctuated less drastically and remained near 520 µs, which is an appropriate value that produced good fairness.

Figure 8 shows the total throughput for the three FQ algorithms at different buffer sizes. As shown in this figure, the total throughput was slightly lower for FQ-VHT with $r = 1$ than for the other algorithms. The holding time plots in Figure 7 demonstrate that the fluctuations in the FQ-VHT holding time were larger with $r = 1$ than with $r = 0.2$. The higher peaks in holding time may have caused the scheduler to wait longer than necessary, resulting in a lower total throughput. However, Figures 9 and 10 show that compared to FQ-VHT with $r = 1$, FQ-VHT with $r = 0.2$ has a higher CUBIC throughput until approximately 10 s have passed. Therefore, when flows with short connection times compete, bandwidth cannot be allocated fairly.

2) Bottleneck link bandwidth of 50 Mb/s: Figure 11 shows the fairness index of the three algorithms at different buffer sizes. The results demonstrate that FQ achieved a lower fairness than FQ-HT or FQ-VHT. As in the case of 25 Mb/s,

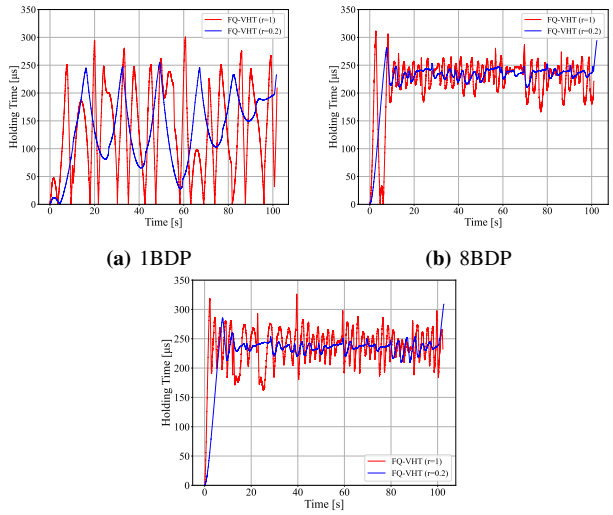
Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on February 04,2026 at 17:00:34 UTC from IEEE Xplore. Restrictions apply.



(a) 1BDP **(b)** 8BDP

(c) 32BDP

Fig. 10: Time Variation of Throughput (25 Mb/s, r=0.2)



(a) 1BDP **(b)** 8BDP

(c) 32BDP

Fig. 12: Time Variation of Holding Time (50 Mb/s)

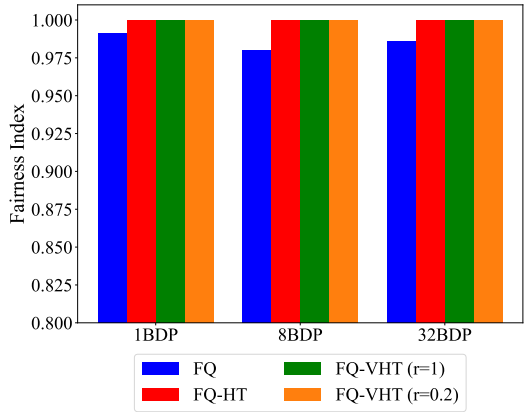


Fig. 11: Fairness Index (50 Mb/s)

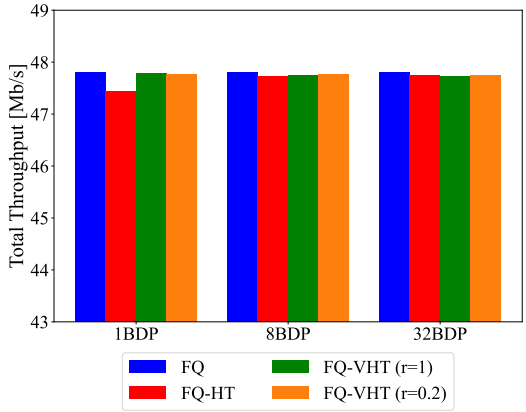


Fig. 13: Total Throughput (50 Mb/s)

FQ was not fair because of the BBR feature of sending packets without accumulating them in the buffer. FQ-HT was more fair at 50 Mb/s because the holding time was sufficient to send packets (240 µs), as shown in Figure 4.

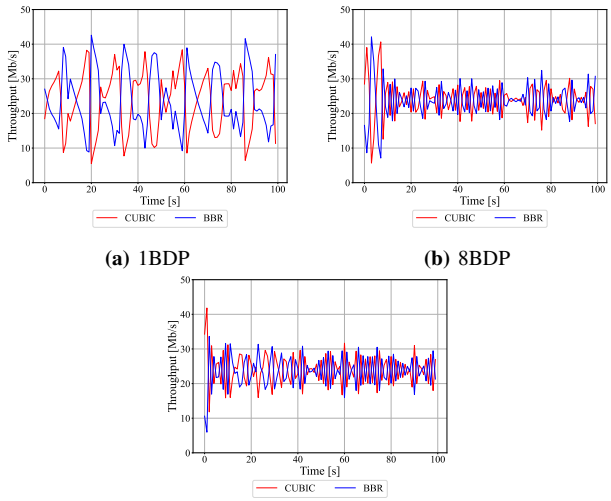
Figure 12 shows the variability in the holding time plotted over the simulation time under different buffer sizes. As shown in Figure 12(a), for a buffer size of 1 BDP, the holding time quickly increased to approximately 250 µs and then showed large oscillations. As in the 25 Mb/s case, this is because the CUBIC transmission rate is lower than the BBR transmission rate at such a small buffer size. Figure 12(b) and (c) shows that under buffer sizes of 8 and 32 BDP, the holding time variation was more moderate and stayed near 240 µs, which is an appropriate holding time.

Figure 13 shows the total throughput for the three FQ algorithms at different buffer sizes. As shown in this figure, the total throughput for FQ-HT at 1 BDP was slightly lower than that of the other algorithms. Figures 4 and 5 show that the appropriate holding time for 1 BDP at 50 Mb/s is approximately 200 µs, which is shorter than the time to send a single packet (240 µs). Therefore, FQ-HT could not transmit fairly. Figure 13 also shows that at 32 BDP, the total throughput for FQ-VHT with $r = 1$ was slightly lower than that of FQ-VHT with $r = 0.2$. The reason for this is that the holding time for FQ-VHT with $r = 1$ was more variable than that for FQ-VHT with $r = 0.2$. However, Figures 14 and 15 show that compared to FQ-VHT with $r = 1$, FQ-VHT with $r = 0.2$ had a higher CUBIC throughput until approximately 10 s passed. Therefore, when flows with short connection times compete, bandwidth is not allocated fairly.

VI. Conclusion

In this study, we proposed a packet scheduling method, FQ-VHT, to fairly send CUBIC and BBR packets stored in different buffers. FQ-VHT calculates and sets the holding time based on the bottleneck link bandwidth and the difference in the number of send packets between buffers. We evaluated the performance of FQ-VHT by changing the buffer size and bandwidth of the bottleneck link and the packet scheduling

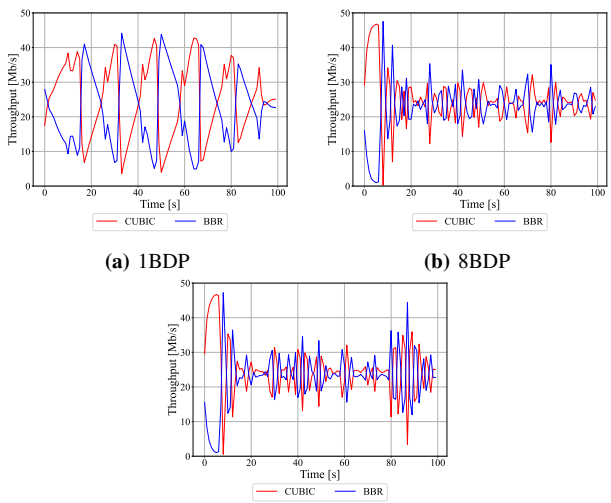
Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on February 04,2026 at 17:00:34 UTC from IEEE Xplore. Restrictions apply.



(a) 1BDP **(b)** 8BDP

(c) 32BDP

Fig. 14: Time Variation of Throughput (50 Mb/s, r=1)



(a) 1BDP **(b)** 8BDP

(c) 32BDP

Fig. 15: Time Variation of Throughput (50 Mb/s, r=0.2)

of the relay nodes, and by performing emulation. The performance evaluation results show that FQ-VHT can achieve a fairness of approximately 1 regardless of the bottleneck link bandwidth and buffer size. Increasing the ratio between the holding time and the difference in the number of send packets from 0.2 to 1 resulted in a shorter time to achieve fairness but a slight decrease in total throughput. In the future, we would like to develop a formula for the holding time that can quickly improve fairness while maintaining total throughput. We would also like to evaluate performance in more complex network environments.

Acknowledgments

This material is based upon work supported by the AFOSR under award number FA2386-24-1-4085.

References

[1] A. Mishra et. al., "The Great Internet TCP Congestion Control Census," ACM Measurement and Analysis of Computing Systems, vol.3, Issue 3, no.45, pp.1-24, Dec. 2019.

[2] Y.-J. Song et.al., "Improvement of Cyclic Performance Variation between TCP BBR and CUBIC," IEEE Asia-Pacific Conference on Communications(APCC), Nov. 2019

[3] K. Ono et.al., "Fairness Improvement of Different TCP Congestion Control Algorithms using P4 Programmable Data Plane," in Proc. IEEE PACRIM2024, Aug. 2024.

[4] Y.-J. Song et. al., "Congestion Window Scaling Method for Inter-protocol Fairness of BBR," IEEE Anual Consumer Communications and Networking Conference(CCNC), March. 2020.

[5] Y.-J. Song et. al., "Understanding of BBRv2: Evaluation and Comparison With BBR v1 Congestion Control" IEEE ACCESS, pp.37131-37145, Feb. 2021.

[6] S. Utsumi et. al., "Improving Inter-Protocol Fairness Based on Estimated Behavior of Competing Flows," IFIP Networking Conference, 2022.

[7] S. Ishikura et. al., "BAR : BBR with Adjusting RTprop for Inter-Protocol Fairness with CUBIC TCP," 2023 IEEE 29th International Symposium on Local and Metropolitan Area Networks (LANMAN),pp. 1-6, July 2023

[8] F. Fejes et. al., "Who will save the internet from the congestion control revolution?," in: International Conference Proceeding Series (ICPS) Proceedings of the 2019 Workshop on Buffer Sizing, Palo Alto, CA, USA, 2019.

[9] E. Kfoury et. al., "P4CCI: P4-based online TCP congestion control algorithm identification for traffic separation," IEEE International Conference on Communications (ICC), Rome, Italy, 2023.

[10] P. Bosshart et. al., "P4: Programming Protocol-Independent Packet Processors," ACM SIGCOM Computer Communication Review, vol44, no. 3, pp. 87-95, 2014.

[11] "Mininet: An Instant Virtual Network on your Laptop (or other PC)," [Online]. Avaliable: https://mininet.org, Accessed on : Nov. 24, 2024.

[12] "BEHAVIORAL MODEL (bmv2)," [Online]. Avaliable: https://github.com/p4lang/behavioral-model, Accessed on : Nov. 24, 2024.

[13] K.R.Jain et. al, "A quantitative measure of fairness and discrimination," Eastern Research Laboratory, Digital Equipment Corporation, Hudson,MA, vol.21, Sep. 1984.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on February 04,2026 at 17:00:34 UTC from IEEE Xplore. Restrictions apply.